

EnCoDaPy: A python Framework for Energy Control and Data Preparation

Martin Altenburger¹, Maximilian Beyer¹, and Paul Seidel¹

¹ Chair of Building Energy Systems and Heat Supply, TUD Dresden University of Technology, Germany

 Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 11 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

EnCoDaPy is a Python framework for energy data processing and control that offers a modular architecture for developing services for data acquisition, calculation, and result reporting. The framework supports multiple interfaces, such as the FIWARE API, MQTT, and file systems, and is specifically designed for applications in energy research. The framework is designed to enable rapid implementation of new algorithms, as these can be integrated into individual components. Core features such as communication, configuration loading and the execution of the calculation cycle are already predefined and ready for use. This significantly reduces the implementation effort and ensures modular reusability.

The central starting point is the use of the open-source FIWARE platform, which provides a key capability for data management ([Blechmann et al., 2023](#); [Cirillo et al., 2019](#)). EnCoDaPy can be integrated within a cloud environment via a FIWARE instance and in decentralized applications using MQTT.

Statement of need

The increasing integration of decentralized energy producers, storage systems, and flexible consumers requires software capable of collecting and processing energy data from heterogeneous sources and exchanging it among distributed components. For research and prototyping applications, data acquisition, configuration management, validation, calculations, and communication often need to be flexibly integrated. Communication interfaces such as MQTT and FIWARE provide important mechanisms for this purpose, but they do not handle the application-specific processing and calculation of energy data.

EnCoDaPy addresses this need as a modular, open-source Python framework for energy management, energy data processing, and control applications. The framework provides a common base service for configuration, data acquisition, calculation, and the transmission of results. Application-specific functions can be implemented, reused, and extended using configurable components. Both current-state data and historical time series can be processed. The framework targets researchers and engineers prototyping energy management and control services such as data analytics.

State of the field

Existing approaches address various aspects of decentralized energy systems, including agent-based control, optimization, simulation, resource allocation, energy trading, and scheduling. The following [Table 1](#) compares selected approaches in terms of their communication interfaces, architectural concepts, data processing capabilities, and open-source status. This

analysis considers two publications describing software frameworks with publicly available implementations (AgentLib (Steffen Eser et al., 2025) and NeuraFlux (Ysaël Desage et al., 2025)) and two scientific articles that describe application-specific multi-agent energy systems ((Blaauwbroek et al., 2015; Davoudi et al., 2024)). The selected references represent categories relevant to EnCoDaPy, including agent-based energy frameworks, simulation-oriented software, and application-specific, multi-agent energy management systems (EMS). This comparison is intended to illustrate the position of EnCoDaPy within the broader software landscape rather than to provide a complete benchmark of individual features.

Table 1: Comparison of EnCoDaPy with related energy-management frameworks.

Software / reference	Communication and integration mechanisms	System design	Data and processing scope	Type
EnCoDaPy	MQTT, FIWARE and file-based interfaces	Modular and extensible framework	current-state and historical time-series data	Open-source framework (Altenburger, Beyer, et al., 2026); BSD-3-Clause license
AgentLib (Steffen Eser et al., 2025)	Local, MQTT, FIWARE (via Plugin)	Modular, component-based agent framework	current-state and historical time-series data, simulation data	Open-source framework (Stefan Eser et al., 2025); BSD-3-Clause license
NeuraFlux (Ysaël Desage et al., 2025)	Internal Python-functions; external integrations described but not identified	Modular agent-based architecture	current-state and historical simulation data	Open-source framework (Ysaël Desage, 2025); Apache-2.0 license
EMS using multi-agent systems (Davoudi et al., 2024)	CANopen	Hierarchical multi-agent system	current simulation data; measurement data not reported	Application-specific research system
Multi-Commodity Smart Energy System (Blaauwbroek et al., 2015)	Agent-based communication; Java Agent Development Framework (JADE)	Distributed multi-agent system	current simulation data; schedule exchange	Application-specific research system

Both EnCoDaPy and AgentLib address some of the same requirements for energy system software, including modular components, data communication and the processing of current-state and historical data. EnCoDaPy was developed as a separate codebase within the N5GEH research consortium and in parallel with related developments such as AgentLib. Its aim is to support configuration-driven, cyclic data processing and control services for real or emulated energy systems. Therefore, the two projects represent parallel developments of related concepts and are not direct extensions or reimplementations of one another.

Although both frameworks use modular architectures and support communication mechanisms or integrations such as MQTT and FIWARE, their design priorities differ. AgentLib offers a general, modular framework for agent-based energy applications which supports control, optimization,

simulation and communication. It also provides an interface to the FIWARE API (Steffen Eser et al., 2025) and supports the integration of simulation models through Functional Mock-up Units (Stefan Eser et al., 2025). EnCoDaPy, in contrast, provides a common runtime environment for configuration, data acquisition, validation, computation, and result transmission, while application-specific functionality is implemented through configurable components. AgentLib primarily organizes applications around interacting agents and simulation- or communication-oriented components, whereas EnCoDaPy primarily organizes service workflows around the recurring runtime functionality of configurable energy-data services.

NeuraFlux is based on a modular, agent-based architecture and supports the processing of current and historical simulation data (Ysaël Desage et al., 2025). Although the publication describes integration with hardware, APIs, and OEM software (Ysaël Desage et al., 2025), the authors' analysis of the publicly accessible repository at commit 4c5ebc6 identified no specific, documented interface for importing real-world data from external systems (Ysaël Desage, 2025). Instead, data exchange in the inspected implementation is performed through internal Python functions. The publication therefore describes a broader intended or conceptual scope than the directly usable external-integration functionality identified in the inspected repository.

Other approaches focus on specific multi-agent energy-management applications, such as resource allocation, energy trading, or schedule generation (Blaauwbroek et al., 2015; Davoudi et al., 2024). The authors did not identify public code repositories for these approaches in the cited publications or during the repository search conducted for this comparison. Taken together, these approaches illustrate how modular and distributed architectures have been applied to energy system control, scheduling, and resource-allocation problems. The selected references focus primarily on agent-based application frameworks, the integration of simulations, or specific energy management tasks, whereas EnCoDaPy emphasizes the combination of a configuration-driven cyclic runtime environment, heterogeneous data interfaces, and reusable application-specific components organized around service workflows.

EnCoDaPy is primarily intended for research and prototyping applications involving data processing and control in real or emulated energy systems. Unlike simulation-oriented frameworks, it does not provide an explicit interface for integrating simulation models. However, simulated or emulated components can be connected via the same interfaces used for real data sources. This allows EnCoDaPy to be used for testing and prototyping without simulation integration being the primary focus.

Software design

Design Overview

EnCoDaPy has a layered, modular architecture which separates data ingestion, processing, and control logic. The core consists of three layers:

- A configuration layer based on Pydantic-validated JSON and environment variables to ensure consistency across deployments and prevent runtime errors caused by malformed configurations.
- A communication layer that abstracts the FIWARE, MQTT, and file-based interfaces behind a unified API to enable interoperability with heterogeneous data sources.
- A component runner that dynamically loads and executes specialized modules (e.g., thermal storage calculators), while maintaining a shared state for real-time data processing.

This design prioritizes both extensibility and real-time capability. Extensibility allows for the addition of custom components without modifying the core, while real-time capability supports variable iteration speeds for simulations and faster-than-real-time studies.

Code Structure

EnCoDaPy is based on a basic (core) service that provides essential functionality for each algorithm, whether for controllers or data preparation. This basic service has been developed as an asynchronous Python application comprising the following subtasks:

- Providing the cyclical service
- Loading and verifying the configuration
- Collecting data from the interfaces
- Running the relevant calculations and providing the results
- Forwarding the data and ensuring the configured information

The basic service enables the creation of a Python service based on this fundamental functionality, with no constraints on the calculation itself. The functions of the basic service are shown as a flow chart in Figure 1.

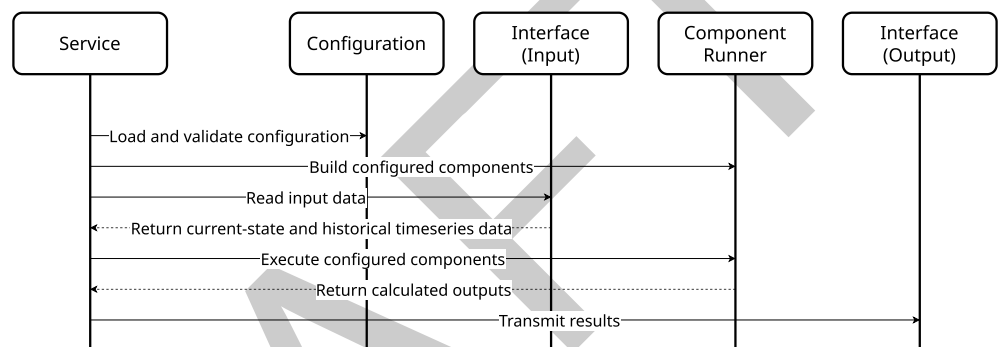


Figure 1: Flow chart of the EnCoDaPy-Service

The core functions are handled by:

▪ ControllerBasicService

This class provides the basic implementation of configuration, interfaces, and cyclic processing. For specific use of the framework, the `calculation()` and `calibration()` functions can be used to integrate custom algorithms for calculation and calibration of that calculation.

EnCoDaPy uses asynchronous execution for communication via MQTT or with the FIWARE API, ensuring that these operations are non-blocking. Additionally, calculation (`calculation()`) and calibration (`calibration()`) are also performed asynchronously. This enables concurrent background processes (e.g., periodic calibration) and flexible user implementations using asynchronous libraries. The architecture is therefore well-suited for EMS that must continuously process data while simultaneously adjusting parameters.

▪ ComponentRunnerService

This second class builds on the `ControllerBasicService` and provides a way to simplify the use of the previously mentioned functions. It introduces the ability to use Components and provides the necessary foundation for doing so. It handles the mapping of input and output data and performs an automatic configuration check. This approach thus facilitates easy reusability, since components can be activated, deactivated, and replaced through configuration and must adhere to a defined structure.

Configuration

The configuration is divided into a main configuration from a file, which is easy to read thanks to its JSON structure, and environmental variables for access credentials. This allows secrets

138 to be handled in a secure way. The main configuration has different parts for each important
139 part of a service based on EnCoDaPy. The structure, including an overview of the function, is
140 shown in Figure 2.

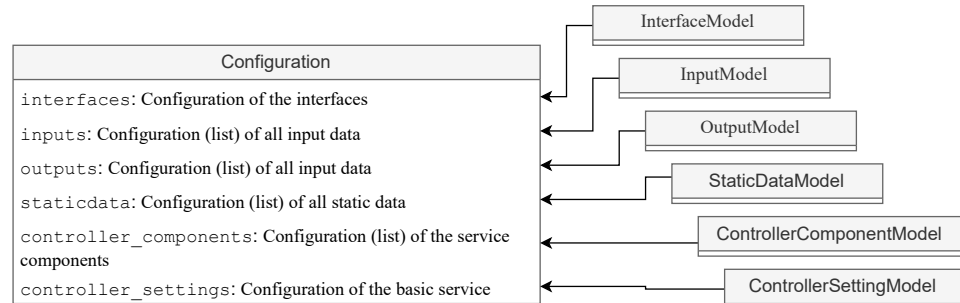


Figure 2: Schema of the basic configuration

141 Verification is performed while loading the configuration using the solutions from Pydantic
142 (Colvin et al., 2024). This means that any configuration-related issues will be logged, and
143 the service will stop if necessary. This ensures that the service will not crash later due to
144 these issues. Using Pydantic (Colvin et al., 2024) BaseModel makes it easy to document the
145 configuration and use the variables in the code without encountering any type issues.

146 Interfaces

147 Collecting data and returning the results can be time-consuming due to the variety of data
148 endpoints. The basic service provides the ability to connect to different interfaces and manage
149 data exchange with these interfaces. Interfaces that are often used are:

- 150 ■ MQTT: Direct exchange via MQTT messages.
- 151 ■ FIWARE: connecting to the context broker of a FIWARE platform ensures the availability
152 of status values and historical time series.
- 153 ■ FILE: Reading or writing data to a file allows data connections to be integrated via file
154 exchange or for default variables to be stored locally.

155 To use these interfaces, they only need to be activated in the configuration, along with the
156 datapoints (input and output data). A combination of interfaces is possible.

157 Components

158 EnCoDaPy's component architecture builds on the basic service, creating a modular, scalable
159 solution for energy data processing. Components perform specific calculations and standardize
160 data processing, reducing implementation effort while enabling flexible exchange. The
161 ComponentRunnerService improves upon the basic flow (see Figure 1) by initializing components
162 from the configuration file and executing them sequentially with automatic data exchange.

163 Using and Creating Components

164 The component runner enables creating a Python service for data preparation or energy
165 management by configuring existing components or building custom ones. To build a custom
166 component, users subclass BasicComponent and implement calculate(); the example below
167 shows a minimal component that aggregates a result.

```

from encodapy.components.basic_component import BasicComponent
from .new_component_config import (

```

```

        NewComponentOutputData
    )

    class NewComponent(BasicComponent):

        def calculate(self) -> None:
            """Perform calculations and set self.output_data"""
            self.output_data = NewComponentOutputData(
                result=self.compute_result()
            )

```

168 EnCoDaPy includes components for calculating thermal storage energy and optimizing model-
169 predictive control, which addresses energy management needs.

170 Thermal Storage Component

171 The Thermal Storage Component provides a temperature-based estimate of the state of charge
172 of a multi-layer thermal storage tank. It models a tank divided into n layers, each with an
173 individual temperature. It then computes the normalized state of charge, φ_{TS} , as a mass-
174 weighted sum of the layer-specific temperatures (see Eq. Equation 1). This estimate assumes
175 a constant heat capacity and is valid for temperatures between the layer-specific reference
176 bounds (ϑ_{TSRi}) and the nominal bounds (ϑ_{TSNi}). Calibration against measurement data
177 or a physical reference model is necessary to achieve exact thermodynamic energy content.

$$\varphi_{TS} = \sum_i^{n_{TS}} \frac{m_i}{m_{TS}} \cdot \frac{\vartheta_{TSi} - \vartheta_{TSRi}}{\vartheta_{TSNi} - \vartheta_{TSRi}} \quad (1)$$

178 Here, m_{TS} denotes the total storage mass, where $m_{TS} = \sum_{i=1}^{n_{TS}} m_i$.

179 FlixOpt Model Component

180 The flixOpt Model Component combines the flixOpt optimization framework (Bumann et
181 al., 2026; Panitz et al., 2022) with EnCoDaPy, allowing for operational optimization without
182 manual setup. The component uses HiGHS as the default solver, though Gurobi is optional and
183 is selected based on the complexity of the model and the required solve speed. The component
184 validates configurations, aggregates incoming time series into an internal DataFrame, and
185 constructs and solves a flixOpt FlowSystem. The results are automatically mapped to EnCoDaPy
186 output data points (e.g., {storage_label}_soc, {converter_label}_thermal_power). The
187 responsibilities between EnCoDaPy and flixOpt are divided as shown in Table 2.

Table 2: Responsibilities between EnCoDaPy and flixOpt in flixOpt Model Component

Task	EnCoDaPy	flixOpt
Input data acquisition	x	
Time series aggregation	x	
Model description	x (via config/JSON)	
Optimization model		x
Solver invocation		x (HiGHS/Gurobi)
Result mapping	x	
External system transfer	x	

188 This component could be used as MPC in building management systems (EMS), where it
189 automatically maps sensor data to the flixOpt inputs and optimization results to the control
190 signals.

Research Impact Statement

The EnCoDaPy framework was developed as part of the N5GEH research consortium and has been used in several satellite projects. The following use cases demonstrate its effectiveness in data analysis and the operational management of building energy systems:

- Virtual sensor technology (domestic hot water (DHW) system):

EnCoDaPy was used to analyze measurement data from a temperature-maintaining band in order to determine the system's operating status based on the DHW temperature and the volume of DHW drawn. Through the connection to the FIWARE platform, historical measurement data was retrieved, and a virtual sensor was implemented in real time using cyclic execution. The analyses were tested through simulations and bench tests achieving a maximal temperature estimation error of 2 K. (Altenburger et al., 2024; Seifert et al., 2025)

- Operations Management (Heat Generation):

Building on the virtual sensor mentioned above, the framework was used in a simulation to control a heat generator for domestic hot water heating, reducing heat losses from the thermal storage by 11.5 % (Altenburger, Seifert, et al., 2026; Seifert et al., 2025). In another project combining a pellet boiler and a heat pump, the framework was used in a field test, to specify the operating states of both heat generators with a cycle time of 10 s (Beyer et al., 2026).

- Cloud/Edge Operation (District Heating + PV):

For a building connected to district heating and equipped with on-site PV generation, EnCoDaPy enabled cloud-based schedule generation and its local implementation at the substation, including data aggregation via the Thermal Storage component. The analyses were conducted using simulation and on a test bench demonstrating schedule-based substation operation and increasing PV self-consumption from 1 % to 2 %. (Seidel et al., 2024)

These projects demonstrate the flexible and practical applicability of EnCoDaPy in various research contexts. As an open-source framework, it promotes reproducibility and interoperability (including through FIWARE integration) in research on building energy systems.

Conclusion

EnCoDaPy is an open-source Python framework designed to enhance the processing and control of energy data, accelerating research and prototyping in energy systems. Its modular architecture, asynchronous execution, and reusable components enable rapid development of applications for data acquisition, calculation, and operational management.

The framework supports FIWARE, MQTT, and file-based interfaces, which enable the integration of heterogeneous data sources and the processing of both real-time and historical data. Pre-built components for thermal storage modeling and model-predictive control reduce implementation effort even further.

EnCoDaPy is designed for research and prototyping in smart grids, building energy management, and decentralized energy systems. Its modularity reduces implementation effort and enables the reuse of algorithms for real or emulated components.

AI usage disclosure

Generative AI tools were used in the development and creation of this work. The tools, their models/versions, and the scope of assistance are listed below.

- 235 ■ **Code validation and review:** Assistance in checking and validating code snippets, including
- 236 as a tool for pull request reviews.
- 237 – GitHub Copilot (model: GPT-4o, hosted via GitHub.com, used 2024–2026)
- 238 – You.com (model: Claude Opus 4, accessed via You.com web interface, 2026)
- 239 – Vibe by Mistral AI (model: Mistral Large 2, accessed 2026)
- 240 ■ **Test development:** Assistance in generating tests, which were subsequently reviewed and
- 241 verified by the authors.
- 242 – GitHub Copilot (model: GPT-4o, hosted via GitHub.com, used 2024–2026)
- 243 – You.com (model: Claude Opus 4, accessed via You.com web interface, 2026)
- 244 ■ **Manuscript:** Assistance in revising the paper’s language and structure.
- 245 – You.com (model: Claude Opus 4, accessed via You.com web interface, 2026)
- 246 – GitHub Copilot (model: GPT-4o, hosted via GitHub.com, used 2024–2026)
- 247 – DeepL (DeepL Write, web application, accessed 2026)
- 248 All AI-generated content (code, tests, text) was reviewed, adapted, and approved by the
- 249 authors, who made all core design decisions.

250 Acknowledgements

251 The authors gratefully acknowledge the financial support provided by the German Federal
 252 Ministry for Economic Affairs and Energy for the research projects “N5GEH-Serv” (grant
 253 number 03EN1030A) and “E3” (grant number 03EN3058C). The sponsors had no role in study
 254 design, software development, or manuscript preparation.

255 References

- 256 Altenburger, M., Beyer, M., & Seidel, P. (2026). *EnCoDaPy - Energy Control and Data*
 257 *Preparation in Python*. Zenodo. <https://doi.org/10.5281/ZENODO.21130839>
- 258 Altenburger, M., Seifert, J., & Röschke, M. (2024, February). *Zustandsdetektion in der*
 259 *Trinkwarmwasserinstallation mittels Temperaturhalteband*. [https://www.tugraz.at/](https://www.tugraz.at/fileadmin/user/_upload/tugrazExternal/f560810f-089d-42d8-ae6d-8e82a8454ca9/files/lf/Session/_F3/635/_LF/_Altenburger/_2.pdf)
 260 [fileadmin/user/_upload/tugrazExternal/f560810f-089d-42d8-ae6d-8e82a8454ca9/files/lf/](https://www.tugraz.at/fileadmin/user/_upload/tugrazExternal/f560810f-089d-42d8-ae6d-8e82a8454ca9/files/lf/Session/_F3/635/_LF/_Altenburger/_2.pdf)
 261 [Session/_F3/635/_LF/_Altenburger/_2.pdf](https://www.tugraz.at/fileadmin/user/_upload/tugrazExternal/f560810f-089d-42d8-ae6d-8e82a8454ca9/files/lf/Session/_F3/635/_LF/_Altenburger/_2.pdf)
- 262 Altenburger, M., Seifert, J., Röschke, M., & Seidel, P. (2026). Optimization of the Operation
 263 of a Domestic Hot Water Heating System Based on Virtual Sensors. In C. Zilio, F.
 264 Busato, L. Mazzarella, & M. Noro (Eds.), *Proceedings of the 15th REHVA HVAC World*
 265 *Congress - CLIMA 2025* (Vol. 788, pp. 710–719). Springer Nature Switzerland. https://doi.org/10.1007/978-3-032-10546-2_66
- 267 Beyer, M., Richter, L., Schinke, L., Seidel, P., Knorr, M., & Seifert, J. (2026). *Field Test of*
 268 *a Hybrid Heating System with Intelligent Control for Flexible, Sustainable Heat Supply.*
 269 *8. Mitteleuropäische Biomassekonferenz CEBC2026 / 8th Central European Biomass*
 270 *Conference CEBC2026*. ISBN: 978-3-9504380-9-3
- 271 Blaauwbroek, N., Nguyen, P. H., Konsman, M. J., Shi, H., Kamphuis, R. I. G., & Kling, W.
 272 L. (2015). Decentralized Resource Allocation and Load Scheduling for Multicommodity
 273 Smart Energy Systems. *IEEE Transactions on Sustainable Energy*, 6(4), 1506–1514.
 274 <https://doi.org/10.1109/TSTE.2015.2441107>
- 275 Blechmann, S., Sowa, I., Schraven, M. H., Streblow, R., Müller, D., & Monti, A. (2023).
 276 Open source platform application for smart building and smart grid controls. *Automation*
 277 *in Construction*, 145, 104622. <https://doi.org/10.1016/j.autcon.2022.104622>
- 278 Bumann, F., Panitz, F., & Stange, P. (2026). *Flixipt*. Zenodo. <https://doi.org/10.5281/ZENODO.18493568>
- 279

- 280 Cirillo, F., Solmaz, G., Berz, E. L., Bauer, M., Cheng, B., & Kovacs, E. (2019). A Standard-
281 Based Open Source IoT Platform: FIWARE. *IEEE Internet of Things Magazine*, 2(3),
282 12–18. <https://doi.org/10.1109/IOTM.0001.1800022>
- 283 Colvin, S., Jolibois, E., Ramezani, H., Badaracco, A. G., Dorsey, T., Montague, D., Matveenko,
284 S., Trylesinski, M., Runkle, S., Hewitt, D., Hall, A., Plot, V., & Maan, D. (2024). *Pydantic*.
285 <https://docs.pydantic.dev/latest/>
- 286 Davoudi, M., Moeini-Aghaie, M., & Mehrtash, M. (2024). Optimal operation of a residential
287 energy hub participating in electricity and heat markets. *Renewable Energy Focus*, 51,
288 100646. <https://doi.org/10.1016/j.ref.2024.100646>
- 289 Desage, Ysaël. (2025). *NeuraFlux*. <https://github.com/ConsultantYD/NeuraFlux>
- 290 Desage, Ysaël, Bouffard, F., & Boulet, B. (2025). NeuraFlux: A scalable and adaptive
291 framework for autonomous data-driven multi-agent power optimization. *Sustainable Energy*,
292 *Grids and Networks*, 44, 101999. <https://doi.org/10.1016/j.segan.2025.101999>
- 293 Eser, Steffen, Storek, T., Wüllhorst, F., Dähling, S., Gall, J., Stoffel, P., & Müller, D. (2025). A
294 modular Python framework for rapid development of advanced control algorithms for energy
295 systems. *Applied Energy*, 385, 125496. <https://doi.org/10.1016/j.apenergy.2025.125496>
- 296 Eser, Stefan, Wüllhorst, F., Storek, T., & Henkel, P. (2025). *AgentLib* (Version v0.8.9).
297 RWTH Aachen University - E.ON Energy Research Center - Institute for Energy Efficient
298 Buildings; Indoor Climate. <https://github.com/RWTH-EBC/AgentLib/>
- 299 Panitz, F., Stange, P., & Behrends, T. (2022). Software-supported Investment Optimization
300 for District Heating Supply Systems. *Proceedings of EuroSun 2022 - ISES and IEA*
301 *SHC International Conference on Solar Energy for Buildings and Industry*, 1–8. <https://doi.org/10.18086/eurosun.2022.04.07>
- 303 Seidel, P., Altenburger, M., & Seifert, J. (2024). *Ein multienergetischer Systemregler für die*
304 *bidirektionale Anbindung an wärmenetze der Zukunft*.
- 305 Seifert, J., Altenburger, M., Schinke, L., Zachrau, R., Lambert, A., Röschke, M., & Grimme, E.
306 (2025). *N5GEH-TWE-Flex - Optimierung und Flexibilisierung von Erzeugungsanlagen zur*
307 *Trinkwassererwärmung: Forschungsbericht*. Hannover : Technische Informationsbibliothek.
308 <https://doi.org/10.34657/26611>